

10 - Listas, mutabilidad

Operaciones de orden superior con listas

Anteriormente se introdujo la noción de programación de orden superior. Puede resultar especialmente práctica con listas, como se muestra a continuación.

Aplicando una función a elementos de la lista

```
1  def apply_to_each(L, f):
2      """Assumes L is a list, f a function
3          Mutates L by replacing each element, e, of L by f(e)"""
4
5      for i in range(len(L)):
6          L[i] = f(L[i])
7
8  L = [1, -2, 3.33]
9
10 print("L =", L)
11 print("Apply abs to each element of L.")
12
13 apply_to_each(L, abs)
14
15 print("L =", L)
16 print("Apply int to each element of", L)
17
18 apply_to_each(L, int)
19
20 print("L =", L)
21 print("Apply squaring to each element of", L)
22
23 apply_to_each(L, lambda x: x ** 2)
24
25 print("L =", L)
```

La función `apply_to_each` se denomina de orden superior porque tiene un argumento que es a su vez una función. La primera vez que se llama, muta `L` aplicando la función incorporada unaria `abs` a cada elemento. La segunda vez que se llama, aplica una conversión de tipo a cada elemento. Y la tercera vez que se llama, reemplaza cada elemento por el resultado de aplicar una función definida usando `lambda`.

```
1  L = [1, -2, 3.33]
2
3  Apply abs to each element of L.
4  L = [1, 2, 3.33]
```

```

5
6 Apply int to each element of [1, 2, 3.33].
7 L = [1, 2, 3]
8
9 Apply squaring to each element of [1, 2, 3].
10 L = [1, 4, 9]

```

Python tiene una función incorporada de orden superior, `map`, que es similar, pero más general, que la función `apply_to_each`. En su forma más simple, el primer argumento de `map` es una función unaria (es decir, una función que sólo tiene un parámetro) y el segundo argumento es cualquier colección ordenada de valores adecuados como argumentos del primer argumento. Se utiliza con frecuencia en lugar de una comprensión de lista. Por ejemplo, `list(map(str, range(10)))` es equivalente a `[str(e) for e in range(10)]`.

La función `map` se utiliza a menudo con un bucle `for`. Cuando se utiliza en un bucle `for`, `map` se comporta como la función `range` en el sentido de que devuelve un valor por cada iteración del bucle. Estos valores se generan aplicando el primer argumento a cada elemento del segundo argumento. Por ejemplo, el código

```

1 for i in map(lambda x: x ** 2, [2, 6, 4]):
2     print(i)

```

imprime

```

1 4
2 36
3 16

```

De forma más general, el primer argumento a mapear puede ser una función de `n` argumentos, en cuyo caso debe ir seguida de `n` colecciones ordenadas posteriores (cada una de la misma longitud). Por ejemplo, el código

```

1 L1 = [1, 28, 36]
2 L2 = [2, 57, 9]
3
4 for i in map(min, L1, L2):
5     print(i)

```

imprime

```

1 1
2 28
3 9

```

✍ Ejercicio manual

Implemente una función que satisfaga la siguiente especificación.

Sugerencia: será conveniente utilizar lambda en el cuerpo de la implementación.

```
1 def f(L1, L2):
2     """L1, L2 lists of same length of numbers
3     returns the sum of raising each element in L1
4     to the power of the element at the same index in L2
5     For example, f([1,2], [2,3]) returns 9"""
6     return sum(map(lambda x, y: x ** y, L1, L2))
```

Devuelvo la suma de los elementos que paso por `map`, el cual mapeara los elementos en `L1` y `L2` en paralelo aplicándoles la función `lambda` que eleva el primer elemento al segundo.

Strings, Tuplas, Rangos y listas

Hemos visto cuatro tipos de secuencias iterables: `str`, `tuple`, `range` y `list`. Son similares en que los objetos de estos tipos pueden ser operados.

Operaciones comunes en tipos de secuencias

- `seq[i]` devuelve el i^{th} elemento en la secuencia.
- `len(seq)` devuelve la longitud de una secuencia.
- `seq1 + seq2` devuelve la concatenación de dos secuencias (no disponible para rangos).
- `n * seq` devuelve la secuencia que repite `seq` `n` veces (no disponible para rangos).
- `seq[start:end]` devuelve un pedazo de la secuencia.
- `e in seq` es `True` si `e` no esta en la secuencia y `False` de lo contrario.
- `e not in seq` es `True` si `e` no esta en la secuencia y `False` de lo contrario.
- `for e in seq` itera sobre los elementos de la secuencia.

Comparación de tipos de secuencias

Tipo	Tipo de elementos	Ejemplos de literales	Mutabilidad
<code>str</code>	Caracteres	<code>""</code> , <code>"a"</code> , <code>"abc"</code>	No
<code>tuple</code>	Cualquier tipo	<code>()</code> , <code>(3,)</code> , <code>("abc", 4)</code>	No
<code>range</code>	Enteros	<code>range(10)</code> , <code>range(1, 10, 2)</code>	No
<code>list</code>	Cualquier tipo	<code>[]</code> , <code>[3]</code> , <code>["abc", 4]</code>	Si

Los programadores de Python tienden a utilizar listas mucho más a menudo que tuplas. Dado que las listas son mutables, pueden construirse de forma incremental durante un cálculo. Por ejemplo, el siguiente código construye incrementalmente una lista que contiene todos los números pares de otra lista.

```

1  even_elems = []
2
3  for e in L:
4      if e % 2 == 0:
5          even_elems.append(e)

```

Dado que las cadenas sólo pueden contener caracteres, son considerablemente menos versátiles que las tuplas o las listas. Por otro lado, cuando se trabaja con una cadena de caracteres, existen muchos métodos incorporados útiles. Ten en cuenta que como las cadenas son **inmutables**, todas ellas devuelven valores y no tienen efectos secundarios.

- `s.count(s1)` cuenta cuántas veces aparece la cadena `s1` en `s`.
- `s.find(s1)` devuelve el índice de la primera aparición de la subcadena `s1` en `s`, y `-1` si no está en `s`.
- `s.rfind(s1)` igual que `find`, pero comienza desde el final de `s` (la "r" de `rfind` significa inverso).
- `s.index(s1)` igual que `find`, pero lanza una excepción si `s1` no está en `s`.
- `s.rindex(s1)` igual que `index`, pero comienza desde el final de `s`.
- `s.lower()` convierte todas las mayúsculas de `s` a minúsculas.
- `s.replace(old, new)` sustituye todas las apariciones de la cadena `old` en `s` con el string `new`.
- `s.rstrip()` elimina los espacios en blanco de `s`.
- `s.split(d)` divide `s` utilizando `d` como delimitador. Devuelve una lista de subcadenas de `s`. Por ejemplo, el valor de `"David Guttag juega al baloncesto".split(" ")` es `['David', 'Guttag', 'juega', 'baloncesto']`. Si `d` es omitido, las subcadenas se separan mediante cadenas arbitrarias de caracteres de espacio en blanco.

Uno de los métodos incorporados más útiles es `split`, que toma dos cadenas como argumentos. El segundo argumento especifica un separador que se utiliza para dividir el primer argumento en una secuencia de subcadenas. Por ejemplo,

```
1 print('My favorite professor-John G.-rocks'.split(' '))
2 print('My favorite professor-John G.-rocks'.split('-'))
3 print('My favorite professor-John G.-rocks'.split('-'))
```

Imprime

```
1 ['My', 'favorite', 'professor-John', 'G.-rocks']
2 ['My favorite professor', '', 'John G.', '', 'rocks']
3 ['My favorite professor', 'John G.', 'rocks']
```

El segundo argumento es opcional. Si se omite ese argumento, la primera cadena se divide utilizando cadenas arbitrarias de caracteres de espacio en blanco (espacio, tabulador, nueva línea, retorno y salto de página).

Ejercicio manual de la lectura

Implemente la función que cumpla las siguientes especificaciones:

```
1 def all_true(n, Lf):
2     """ n is an int
3         Lf is a list of functions that take in an int and return a Boolean
4         Returns True if each and every function in Lf returns True when called
5         with n as a parameter. Otherwise returns False.
6     """
7     for f in Lf:
8         if not f(n):
9             return False
10    return True
11
12 # Examples:
13 all_true() # prints 6
```

Básicamente recorro `Lf` y si alguna función `f(n)` no es verdadera, entonces devuelvo `False`.